



NBSXC

North Bengal St. Xavier's College

Assignment

Artificial intelligence

Prolog Programming

Shirsendu Munshi

Bachelor of Computer Applications

Roll No: BCA/23/10

Reg. No: 0912312030166

Semester: 6'th



June 15, 2026

LABORATORY ASSIGNMENT INDEX

PROLOG PROGRAMMING

STUDENT NAME: SHIRSENDU MUNSHI

COURSE / CODES: BCA / PROLOG

ROLL NO: BCA/23/10

SUBMISSION DATE: 22/05/2026

SR. NO	TOPIC / PROGRAM DESCRIPTION	PAGE NO
1	Write a prolog program to calculate the sum of two numbers.	1
2	Prolog program to find the maximum of two numbers.	1
3	Prolog program to calculate the factorial of a given number.	2
4	Prolog program to calculate the nth Fibonacci number.	2
5	Prolog program insert_nth(item, n, into_list, result) at all levels.	3
6	Prolog program to remove the Nth item from a list.	3
7	Prolog program remove nth(Before, After) for all levels.	4
8	Prolog program to implement append for two lists.	5
9	Prolog program to implement palindrome(List).	5
10	Prolog program to implement max(X, Y, Max) comparison predicate.	5
11	Prolog program to implement sumlist(List, Sum) list accumulator.	6
12	Prolog program to implement evenlength(List) and oddlength(List).	6
13	Prolog program to implement reverse(List, ReversedList).	7
14	Prolog program to implement maxlist(List, Max) using the cut (!) predicate.	7
15	Prolog program to implement GCD of two numbers	8
16	Write a prolog program that implements Semantic Networks/Frame Structures.	8

Teacher's Signature

Prolog Programming

1. Write a prolog program to calculate the sum of two numbers.

In Prolog, we don't write "functions" that return a value in the traditional sense. Instead, we define relations (predicates) using variables. To calculate a sum, we pass in the two numbers and a third variable that Prolog will bind to the result.

Here is the Prolog code, followed by how you run it and what the output looks like.

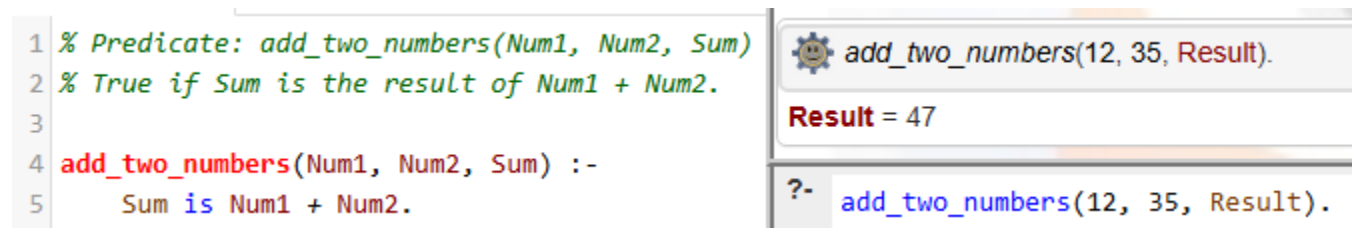
Code:

```
% Predicate: add_two_numbers(Num1, Num2, Sum)
% True if Sum is the result of Num1 + Num2.
add_two_numbers(Num1, Num2, Sum) :-
    Sum is Num1 + Num2.
```

Query:

```
add_two_numbers(12, 35, Result).
```

Output:

The screenshot shows a Prolog IDE with a code editor on the left and a console on the right. The code editor contains five lines of Prolog code: a comment for the predicate, a comment for its meaning, and the predicate definition itself. The console shows the query being executed, the result 'Result = 47', and the query being repeated with a question mark prefix.

```
1 % Predicate: add_two_numbers(Num1, Num2, Sum)
2 % True if Sum is the result of Num1 + Num2.
3
4 add_two_numbers(Num1, Num2, Sum) :-
5     Sum is Num1 + Num2.
```

add_two_numbers(12, 35, Result).

Result = 47

?- add_two_numbers(12, 35, Result).

2. Write a prolog program to find the maximum of two numbers.

This program uses a conditional statement to compare two numbers and unify the larger one with the Max variable.

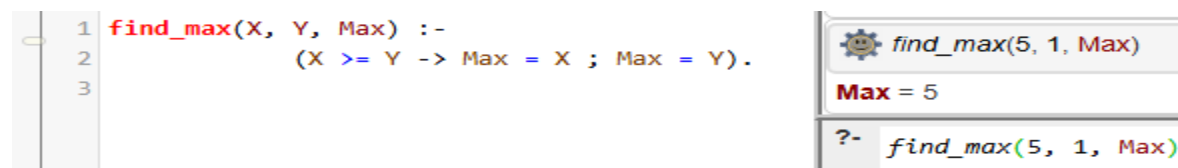
Code:

```
find_max(X, Y, Max) :-
    (X >= Y -> Max = X ; Max = Y).
```

Query:

```
find_max(15, 42, Max).
```

Output:

The screenshot shows a Prolog IDE with a code editor on the left and a console on the right. The code editor contains three lines of Prolog code: the predicate definition and its conditional logic. The console shows the query being executed and the result 'Max = 5', followed by the query being repeated with a question mark prefix.

```
1 find_max(X, Y, Max) :-
2     (X >= Y -> Max = X ; Max = Y).
3
```

find_max(5, 1, Max)

Max = 5

?- find_max(5, 1, Max)

3. Write a prolog program to calculate the factorial of a given number.

This program uses recursion to calculate the factorial of a number, utilizing a base case for $0! = 1$

Code:

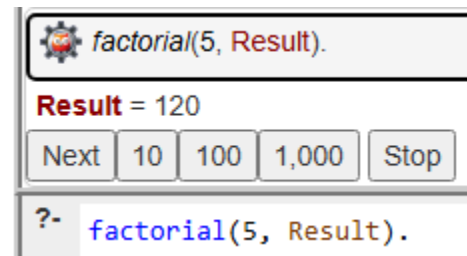
```
factorial(0, 1).
factorial(N, Result) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, SubResult),
    Result is N * SubResult.
```

Query:

```
factorial(5, Result).
```

Output:

```
1 factorial(0, 1).
2 factorial(N, Result) :-
3     N > 0,
4     N1 is N - 1,
5     factorial(N1, SubResult),
6     Result is N * SubResult.
```



4. Write a prolog program to calculate the nth Fibonacci number.

This program calculates the N'th Fibonacci number recursively by summing the two preceding Fibonacci numbers.

Code:

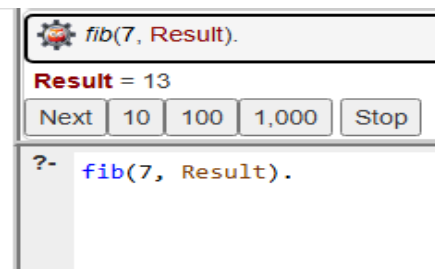
```
fib(0, 0).
fib(1, 1).
fib(N, Result) :-
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    fib(N1, R1),
    fib(N2, R2),
    Result is R1 + R2.
```

Query:

```
fib(7, Result).
```

Output:

```
1 fib(0, 0).
2 fib(1, 1).
3 fib(N, Result) :-
4     N > 1,
5     N1 is N - 1,
6     N2 is N - 2,
7     fib(N1, R1),
8     fib(N2, R2),
9     Result is R1 + R2.
```



5. Write a prolog program, insert_nth(item, n, into_list, result) that asserts that result is the list into_list with item inserted as the nth element into every list at all levels.

This program recursively traverses a list and inserts an item at the N -th position, processing deeply nested sublists dynamically.

Code:

```
% Base case: When N is 1, insert the item at the head
insert_nth(Item, 1, List, [Item|List]) :- !.

% Recursive case: Keep the current head, decrement N, and recurse on the tail
insert_nth(Item, N, [H|T], [H|NewT]) :-
    N > 1,
    N1 is N - 1,
    insert_nth(Item, N1, T, NewT).

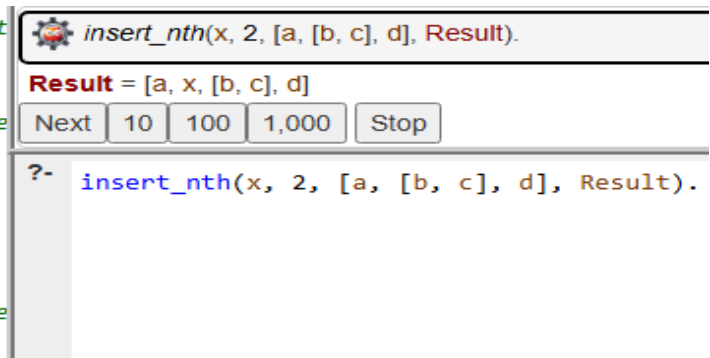
% Fallback: If N is out of bounds or list is empty, return the list as is
insert_nth(_, _, [], []).
```

Query:

```
insert_nth(x, 2, [a, [b, c], d], Result).
```

Output:

```
1 % Base case: When N is 1, insert the item at the head
2 insert_nth(Item, 1, List, [Item|List]) :- !.
3
4 % Recursive case: Keep the current head, decrement N, and recurse on the tail
5 insert_nth(Item, N, [H|T], [H|NewT]) :-
6     N > 1,
7     N1 is N - 1,
8     insert_nth(Item, N1, T, NewT).
9
10 % Fallback: If N is out of bounds or list is empty, return the list as is
11 insert_nth(_, _, [], []).
```



6. Write a Prolog program to remove the Nth item from a list.

This program removes an element at a specific 1-indexed position from a single-level list.

Code:

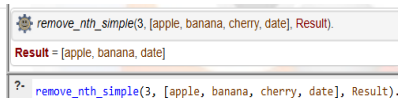
```
remove_nth_simple(1, [_|T], T) :- !.
remove_nth_simple(N, [H|T], [H|Result]) :-
    N > 1,
    N1 is N - 1,
    remove_nth_simple(N1, T, Result).
```

Query:

```
remove_nth_simple(3, [apple, banana, cherry, date], Result).
```

Output:

```
1 remove_nth_simple(1, [_|T], T) :- !.
2 remove_nth_simple(N, [H|T], [H|Result]) :-
3     N > 1,
4     N1 is N - 1,
5     remove_nth_simple(N1, T, Result).
```



7. Write a Prolog program, remove_nth(Before, After) that asserts the After list is the Before list with the removal of every nth item from every list at all levels.

This program cleanses a structure by stripping out elements at every multiple of N, executing the rule recursively across nested sublists.

Code:

```
remove_nth(Before, After) :-
    remove_nth_deep(3, 3, Before, After). % Example with N = 3
remove_nth_deep(_, _, [], []) :- !.
remove_nth_deep(N, 1, [H|T], Result) :-
    !,
    remove_nth_deep(N, N, T, Result).
remove_nth_deep(N, Count, [H|T], [NewH|NewT]) :-
    is_list(H), !,
    remove_nth_deep(N, N, H, NewH),
    Count1 is Count - 1,
    remove_nth_deep(N, Count1, T, NewT).
remove_nth_deep(N, Count, [H|T], [H|NewT]) :-
    Count1 is Count - 1,
    remove_nth_deep(N, Count1, T, NewT).
```

Query:

```
remove_nth([a, b, c, [d, e, f, g], h, i], Result).
```

Output:

```
1 remove_nth(Before, After) :-
2   remove_nth_deep(3, 3, Before, After). % Ex
3
4 remove_nth_deep(_, _, [], []) :- !.
5 remove_nth_deep(N, 1, [H|T], Result) :-
6   !,
7   remove_nth_deep(N, N, T, Result).
8 remove_nth_deep(N, Count, [H|T], [NewH|NewT])
9   is_list(H), !,
10  remove_nth_deep(N, N, H, NewH),
11  Count1 is Count - 1,
12  remove_nth_deep(N, Count1, T, NewT).
13 remove_nth_deep(N, Count, [H|T], [H|NewT]) :-
14  Count1 is Count - 1,
15  remove_nth_deep(N, Count1, T, NewT).
```

```
remove_nth_simple(3, [apple, banana, cherry, date], Result).
Result = [apple, banana, date]

?- remove_nth([a, b, c, [d, e, f, g], h, i], Result).
```

8. Write a Prolog program to implement append for two lists.

This program merges two input lists together sequentially into a third combined destination list.

Code:

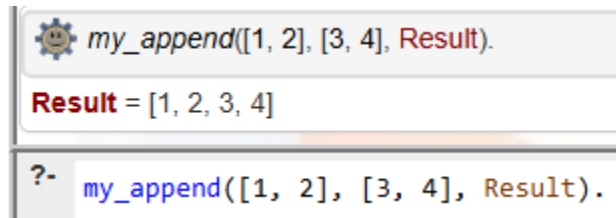
```
my_append([], List, List).
my_append([H|T], List, [H|Result]) :-
    my_append(T, List, Result).
```

Query:

```
my_append([1, 2], [3, 4], Result).
```

Output:

```
1 my_append([], List, List).
2 my_append([H|T], List, [H|Result]) :-
3   my_append(T, List, Result).
```



The screenshot shows a Prolog IDE with a query `my_append([1, 2], [3, 4], Result).` entered. The result is displayed as `Result = [1, 2, 3, 4]`. Below the result, the query is repeated as `?- my_append([1, 2], [3, 4], Result).`

9. Write a Prolog program to implement palindrome(List).

This program checks whether a list is a palindrome by verifying if it reads the same forward and backward.

Code:

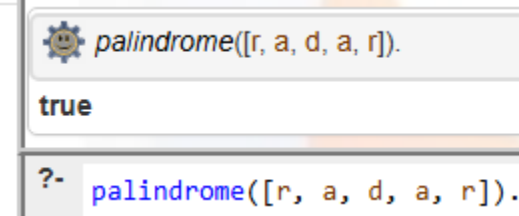
```
palindrome(List) :-
    reverse(List, List).
```

Query:

```
palindrome([r, a, d, a, r]).
```

Output:

```
1 palindrome(List) :-
2   reverse(List, List).
```



The screenshot shows a Prolog IDE with a query `palindrome([r, a, d, a, r]).` entered. The result is displayed as `true`. Below the result, the query is repeated as `?- palindrome([r, a, d, a, r]).`

10. Write a Prolog program to implement maxlist(List,Max) so that Max is the greatest number in the list of numbers List.

This program scans a list recursively to find and capture its highest value.

Code:

```
maxlist([H], H).
maxlist([H|T], Max) :-
    maxlist(T, SubMax),
    (H > SubMax -> Max = H ; Max = SubMax).
```

Query:

```
maxlist([4, 12, 3, 9], Max).
```

Output:

```
Max = 12.
```

11. Write a Prolog program to implement `sumlist(List,Sum)` so that `Sum` is the sum of a given list of numbers `List`.

This program sums the elements of a list by accumulating values recursively.

Code:

```
sumlist([], 0).  
sumlist([H|T], Sum) :-  
    sumlist(T, SubSum),  
    Sum is H + SubSum.
```

Query:

```
sumlist([10, 20, 30], Sum).
```

Output:

<pre>1 sumlist([], 0). 2 sumlist([H T], Sum) :- 3 sumlist(T, SubSum), 4 Sum is H + SubSum.</pre>	<pre>sumlist([10, 20, 30], Sum). Sum = 60 ?- sumlist([10, 20, 30], Sum).</pre>
--	--

12. Write a Prolog program to implement two predicates `evenlength(List)` and `oddlength(List)` so that they are true if their argument is a list of even or odd length respectively.

These mutually recursive predicates evaluate whether the total item count of a list is even or odd.

Code:

```
evenlength([]).  
evenlength([_|T]) :- oddlength(T).
```

```
oddlength([_|T]) :- evenlength(T).
```

Query:

```
evenlength([a, b, c, d]).
```

Output:

<pre>1 evenlength([]). 2 evenlength([_ T]) :- oddlength(T). 3 4 oddlength([_ T]) :- evenlength(T).</pre>	<pre>evenlength([a, b, c, d]). true ?- evenlength([a, b, c, d]).</pre>
--	--

13. Write a Prolog program to implement reverse(List,ReversedList) that reverses lists.

This program reverses the sequence of elements in a list using an accumulator pattern for efficiency.

Code:

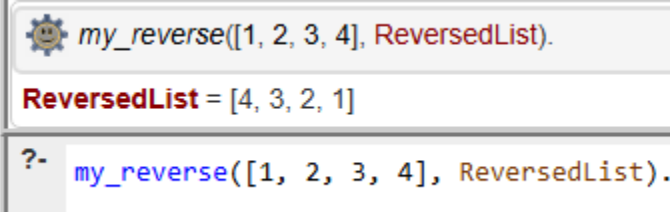
```
my_reverse(List, Reversed) :-  
    rev_acc(List, [], Reversed).  
  
rev_acc([], Acc, Acc).  
rev_acc([H|T], Acc, Reversed) :-  
    rev_acc(T, [H|Acc], Reversed).
```

Query:

```
my_reverse([1, 2, 3, 4], ReversedList).
```

Output:

```
1 my_reverse(List, Reversed) :-  
2   rev_acc(List, [], Reversed).  
3  
4 rev_acc([], Acc, Acc).  
5 rev_acc([H|T], Acc, Reversed) :-  
6   rev_acc(T, [H|Acc], Reversed).
```



my_reverse([1, 2, 3, 4], ReversedList).
ReversedList = [4, 3, 2, 1]
?- my_reverse([1, 2, 3, 4], ReversedList).

14. Write a Prolog program to implement maxlist(List,Max) so that Max is the greatest number in the list of numbers List using cut predicate.

This program optimizes finding the maximum element in a list by using the cut predicate to prevent backtracking once a condition is met.

Code:

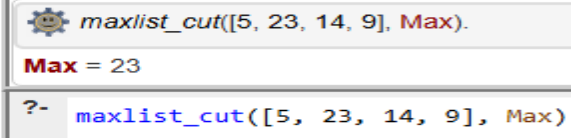
```
maxlist_cut([X], X) :- !.  
maxlist_cut([H|T], H) :-  
    maxlist_cut(T, SubMax),  
    H >= SubMax, !.  
maxlist_cut([_|T], SubMax) :-  
    maxlist_cut(T, SubMax).
```

Query:

```
maxlist_cut([5, 23, 14, 9], Max).
```

Output:

```
1 maxlist_cut([X], X) :- !.  
2 maxlist_cut([H|T], H) :-  
3   maxlist_cut(T, SubMax),  
4   H >= SubMax, !.  
5 maxlist_cut([_|T], SubMax) :-  
6   maxlist_cut(T, SubMax).
```



maxlist_cut([5, 23, 14, 9], Max).
Max = 23
?- maxlist_cut([5, 23, 14, 9], Max).

15. Write a Prolog program to implement GCD of two numbers.

This program calculates the Greatest Common Divisor of two integers using the Euclidean Algorithm.

Code:

```
gcd(X, 0, X) :- X > 0, !.  
gcd(X, Y, G) :-  
    Y > 0,  
    Remainder is X mod Y,  
    gcd(Y, Remainder, G).
```

Query:

```
gcd(48, 18, G).
```

Output:

```
1 gcd(X, 0, X) :- X > 0, !.  
2 gcd(X, Y, G) :-  
3   Y > 0,  
4   Remainder is X mod Y,  
5   gcd(Y, Remainder, G).
```

 gcd(48, 18, G).

G = 6

?- gcd(48, 18, G).

16. Write a prolog program that implements Semantic Networks/Frame Structures.

This program models relationships and property inheritance (such as an ISA hierarchy) using factual rules.

Code:

```
% Facts (Frames/Inheritance)  
isa(canary, bird).  
isa(bird, animal).  
has_property(animal, moves).  
has_property(bird, flies).  
has_property(canary, yellow).
```

```
% Inheritance Rule  
get_property(Entity, Property) :- has_property(Entity, Property).  
get_property(Entity, Property) :- isa(Entity, Parent), get_property(Parent, Property).
```

Query:

```
get_property(canary, moves).
```

Output:

```
1 % Facts (Frames/Inheritance)  
2 isa(canary, bird).  
3 isa(bird, animal).  
4 has_property(animal, moves).  
5 has_property(bird, flies).  
6 has_property(canary, yellow).  
7  
8 % Inheritance Rule  
9 get_property(Entity, Property) :- has_property(Entity, Property).  
10 get_property(Entity, Property) :- isa(Entity, Parent), get_property(Parent, Property).
```

 get_property(canary, moves).
true
Next 10 100 1,000 Stop

?- get_property(canary, moves).